Zoo Management System Testing

Testing Approach & Test Cases

Testing Approach

My approach to testing the management application initially consisted of multi-layered testing to ensure a positive user experience and data integrity. Using black box testing I focused on the functional requirements via the command line interface, ensuring a variety of user inputs (normal, boundary and invalid) were handled correctly for adding and editing animals. This ensured the program would not crash or mishandle inputs from the user from a variety of inputs, which can lead to data corruption or loss.

Additionally, I then proceeded to test other functions such as generating a report, searching, viewing all animals and performing daily care to ensure they functioned as the requirements intended. When testing, they would pass however I did not like how data such as the report and daily care were presented initially which led me to improving the usability by creating line breaks using println() and proper structure to display the requested details.

This testing was followed by unit testing to verify the logic of the Animal abstract class and the subclasses in an isolated environment. Unit testing allowed me to efficiently test my application without having to run the main application over and over and risking the possibility of corrupting my data from crashes or intentional misinput. File saving and data persistence was verified by manual inspection of the ZooStorage.txt file.

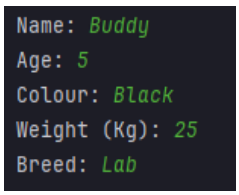
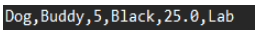
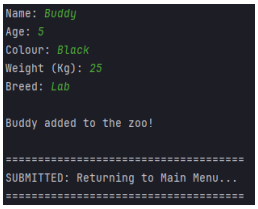
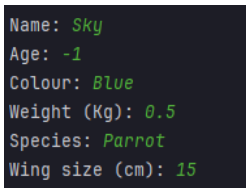
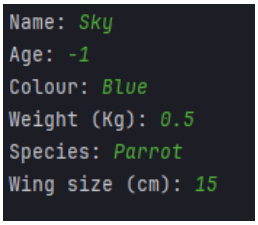
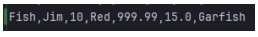
Data Integrity & Security

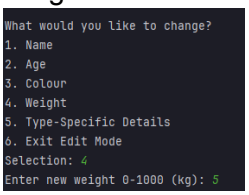
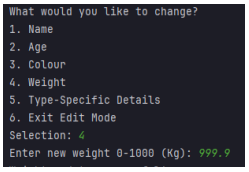
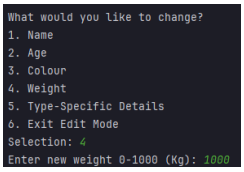
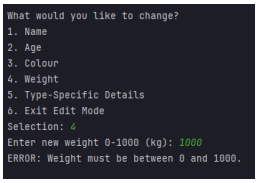
To ensure no null data is added into any field, I have added .isEmpty() functions and compared numerical input values with 0. This allows the user to instantly get feedback on invalid fields and it forces them to enter data again in order to even save it to the file. Additionally, the application informs the user that there is no file on first startup and always creates one upon exiting and saving any data.

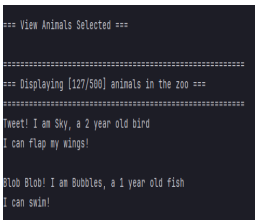
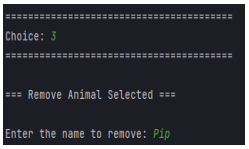
System Test Case Summary

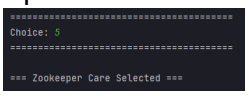
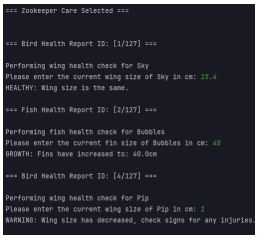
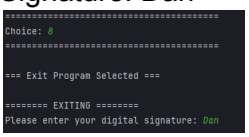
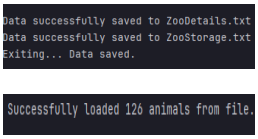
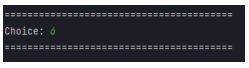
Total Tests: 17
Passed: 17
Failed: 1, Initially failed TC-5 because weight of 1000 was accepted. Fixed the logic to be < 1000 instead of <=
Conclusion: The application is stable for deployment with all requirements tested.

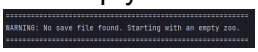
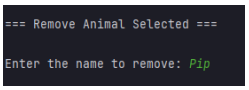
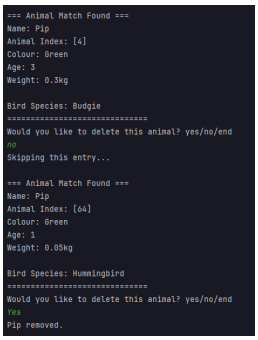
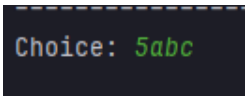
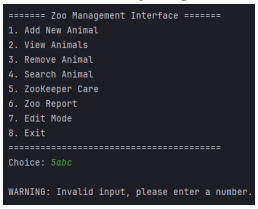
System Test Cases

TC	Description	Test Data	Expected Results	Actual Results	Pass/Fail
TC-1	Add Animal (Normal Condition) Determines data from the superclass and subclass can be added.	Dog: "Buddy" Age: 5 Colour: “Black” Weight: 25 Breed: “Lab” 	1. “Animal successfully added” Message displayed to the user. 2. Saves to file upon exit.	The system confirmed “Buddy added to the zoo!”  	**Pass**
TC-2	Add Animal (Invalid, Boundary Number) Determines data from the superclass and subclass are validated.	Bird: “Sky” Age: -1 Colour: “Blue” Weight: 0.5 Species: “Parrot” Wing Size: 15 	1. “Invalid data entry. Please check all fields filled in. Age and weight fields have to be greater than 0.” Message displayed to the user. 2. Does not save to file.	The system displayed the error message and did not save the data. 	**Pass**
TC-3	Add Animal (Valid, Boundary Number) Determines data from the superclass and subclass are validated.	Fish: “Jim” Age: 10 Colour: “Red” Weight: 999.99 Species: “Garfish” Fin Size: 15 	1. “Animal successfully added” Message displayed to the user. 2. Saves to file upon exit.	The system confirmed “Jim was added to the zoo!”  	**Pass**

TC-4	Edit Animal (Normal Boundary) Determines data from the superclass and subclass can be edited.	Name: Sky Decision: Yes Selection: 4 Weight: 5 	1. “Weight update successful!” Displayed message command line interface.	“Weight update successful” displayed and weight saved to ZooStorage.txt 	**Pass**
TC-5	Edit Animal (Boundary: Valid Weight) Determines edited data is validated.	Name: Sky Decision: Yes Selection: 4 Weight: 999.9 	1. “Weight update successful!” Displayed message command line interface.	“Weight update successful” displayed and weight saved to ZooStorage.txt 	**Pass**
TC-6	Edit Animal (Boundary: Invalid Weight) Determines edited data is validated.	Name: Sky Decision: Yes Selection: 4 Weight: 1000 	1. “ERROR: Weight must be between 0 and 1000kg. Update failed.” 2. Data not saved to ZooStorage.txt	Error message showing weight must be between 0 and 1000 Nothing submitted. 	**Fail (1) Logic error, Fixed by removing equal to.** **Pass (2)**

TC-7	View Animals (Functionality Test) Determines that all animals from all subclasses can make their makeSound() methods correctly.	Choice: 2 	1. Play all animal sounds for each animal.	Message showing the amount of animals, all sounds and interface methods. 	**Pass**
TC-8	Remove Animals (Functionality Test) Determines that specific animal data can be removed from the application.	Choice: 3 Name: Pip 	1. Animals with the matching name will display for the user to remove it.	The animal was removed from the zoo, and will be deleted from file upon exit. 	**Pass**
TC-9	Search Animal By Colour (Functionality Test) Determines that specific animal data can be queried by colour	Choice: 4 Colour: Green 	1. Animals with their full details and sound will be displayed. 2. An option asking if this is correct.	Animals with the matching colour are shown to the user and they can select if that record is correct. 	**Pass**

TC-10	Search Animal By Name (Functionality Test) Determines that specific animal data can be queried by name	Choice: 4 Name: Pip 	1. Animals with their full details will be displayed. 2. An option asking if this is correct. the next animal.	Animals with the matching name are displayed to the user 	**Pass**
TC-11	Daily Care on An Animals (Functionality Test) Determines that the ZooKeeper can perform care on all animals.	Choice: 5 Input: 15.4 Input: 40 Input: 1 	1. Animal health is indicated by if their size stays the same, increases or decreases.	Daily care is offered for all animals and compares their instance variables. 	**Pass**
TC-12	File Persistence (Save/Load) Determines that the file will save data into a file on exit and load data on startup.	Starting the Program Choice: 8 Signature: Dan 	1. Data will save to the ZooStorage.txt 2. Data will be loaded on running the program.	Running the program loads and saves to ZooStorage.txt ZooDetails.txt  	**Pass**
TC-13	Zoo Report (Functionality Test) Determines that all stored data can be queried and data can be selected to be presented.	Choice: 6 	1. Count for all the animals will be displayed 2. The most common colour of all animals will be displayed.	Report Displayed. 	**Pass**

TC-14	New Zoo Loading From File Determining how the application will react when it is first loaded.	Starting the program without a ZooStorage.txt File	1. A warning message will appear stating that no file is found and will start with an empty zoo.	No data is loaded. “Warning: No save file found. Starting with empty zoo” 	**Pass**
TC-15	Duplicate Name Remove Animal Determining what happens if the user attempts to remove an animal with the same name.	Choice: 3 Name: Pip Choice: no Choice: Yes 	1. The user will be able to choose which animal to delete based off their specified details 2. That specific animal will only be deleted.	The specific animal selected has been deleted 	**Pass**
TC-16	Case Sensitivity (Functionality) Using searchAnimal as an example.	Choice: 4 Name: jEweL 	1. Regardless of capitalisation, Jewel will be found.	Animal found successfully. 	**Pass**
TC-17	Invalid Menu Selection Ensure the application correctly handles user misinput in the menu.	Choice: 5abc 	1. The menu will warn the user of incorrect data entry and allow them to try again.	Warning message of invalid input, the user can try again. 	**Pass**

Unit Testing & Summary Report

Purpose

Unit tests were created to verify the makeSound() polymorphism method alongside the creation of classes and their interaction with the main abstract class. I tested if the Dog, Bird and Fish subclasses would correctly return their correct specific sound strings overridden from their subclass. This environment allows me to test code without it writing to the storage files.

Unit Testing Code

```
public class DogTest {
    low complexity (7%)
    Rename usages
    @Test
    public void testAnimalSound() {
        Dog dog = new Dog( name: "Jimmy", age: 10, colour: "Red", weight: 100.0, breed: "Maple");
        String expected = "Woof! I am Jimmy, a 10 year old dog";

        if (expected.equals(dog.makeSound())) {
            System.out.println("Dog Sound Test Passed");
        }
    }
}
```

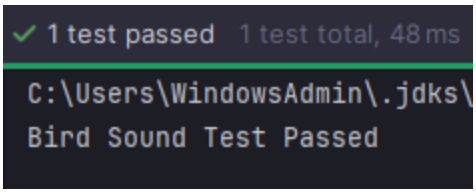
```
public class FishTest {
    low complexity (7%)
    Rename usages
    @Test
    public void testAnimalSound() {
        Fish fish = new Fish( name: "Bubbles", age: 1, colour: "Orange", weight: 0.1, finSize: 2.5, species: "Goldfish");
        String expected = "Blob Blob! I am Bubbles, a 1 year old fish";

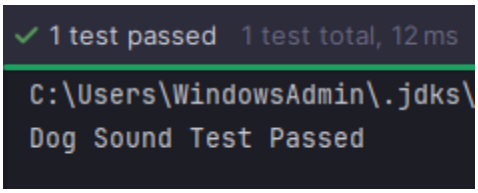
        if (expected.equals(fish.makeSound())) {
            System.out.println("Fish Sound Test Passed");
        }
    }
}
```

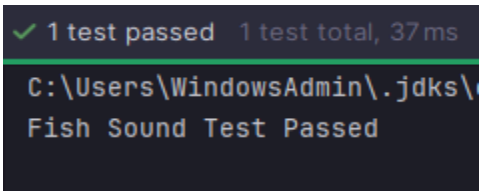
```
public class BirdTest {
    low complexity (7%)
    Rename usages
    @Test
    public void testAnimalSound() {
        Bird bird = new Bird( name: "Sky", age: 2, colour: "Blue", weight: 0.5, wingSize: 15.4, species: "Parrot");
        String expected = "Tweet! I am Sky, a 2 year old bird";

        if (expected.equals(bird.makeSound())) {
            System.out.println("Bird Sound Test Passed");
        }
    }
}
```

Unit Results







Result Summary

These tests show the correct implementation of polymorphism as each animal overrides the makeSound() from the Animal superclass. Dogs, fish and birds display their own unique sounds and store their own independent subclass values.

Furthermore, this test shows that the makeSound() method properly formats variables such as name and age into their own customised sentence.

The data state of animals is also checked as a new object is successfully created. This means that the constructors and getter methods are fully functional.

Requirement Summary Table

Requirement ID:	Implemented (Yes/No)	Associated Test IDs
1. Handle data of three different types of animals	**Yes**	**TC-1, TC-2, TC-3**
2. Record name, age, colour, and weight.	**Yes**	**TC-1, TC-2, TC-3**
3. Introduce an additional instance variable.	**Yes**	**TC-1, TC-2, TC-3**
4. Use a method that returns an animal noise	**Yes**	**TC-7, TC-9, TC-10**

5. Add, remove or modify animals	**Yes**	**TC-1, TC-2, TC-3, TC-4, TC-5, TC-6, TC-8**
6. Daily care for animals	**Yes**	**TC-11**
7. Animal Report	**Yes**	**TC-14**
8. View all animals	**Yes**	**TC-7**
9. Save all data into a text file.	**Yes**	**TC-12, TC-14**
10. Re-populate the application using previous stored data.	**Yes**	**TC-12, TC-14**
11. Search for an animal and display their sound and details	**Yes**	**TC-9, TC-10, TC-16**